EBU5408

Digital Audio Fundamentals

MIDI and Sound Synthesis

Dr. Marie-Luce Bourguet

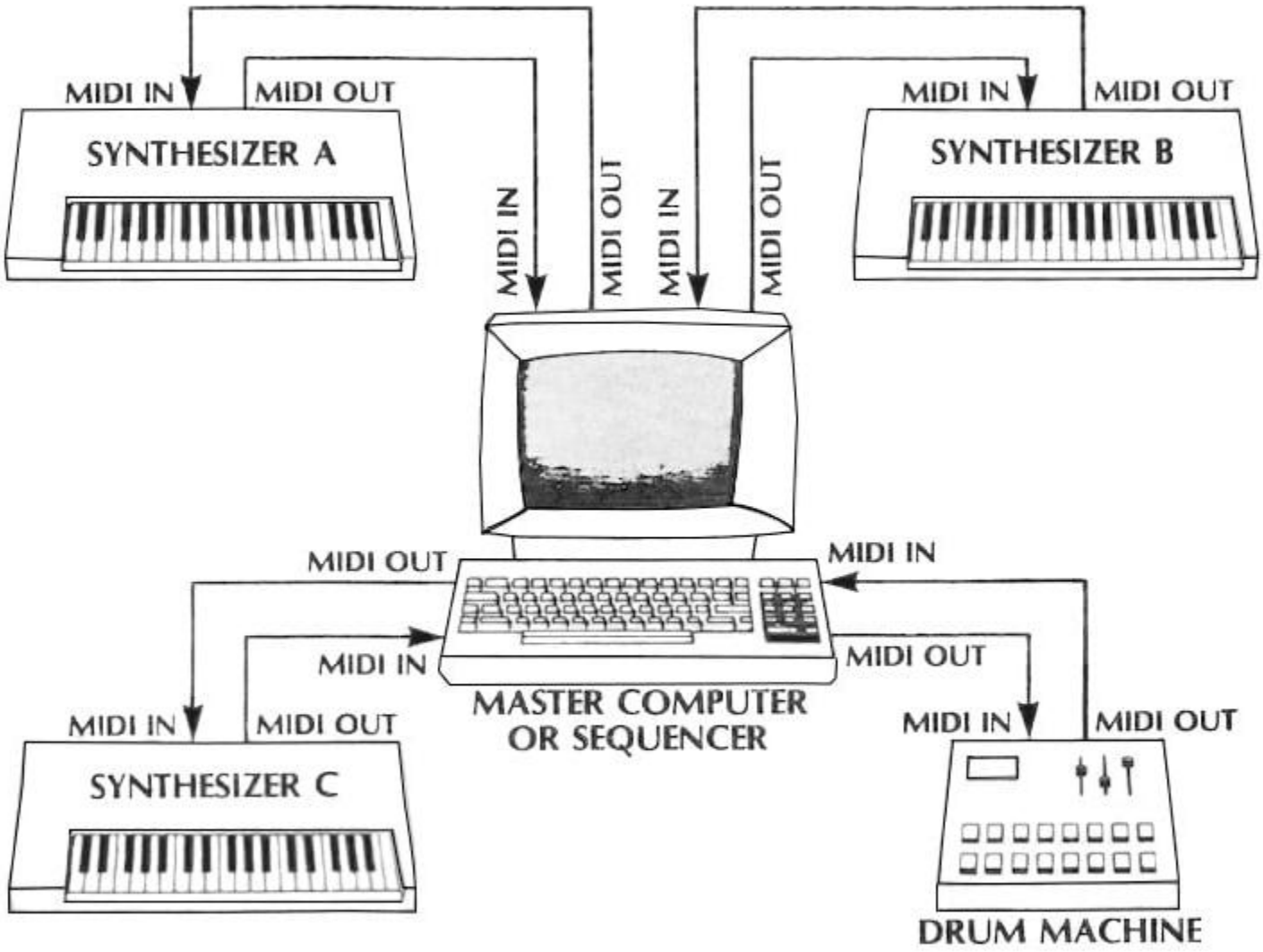
marie-luce.bourguet@qmul.ac.uk

Agenda

- Overview of MIDI and Sound Synthesis
- Understanding MIDI Messages
- Hardware Aspects of MIDI
- Basics of Sound Synthesis
- Types of Synthesis

What is MIDI?

- MIDI = Musical Instrument Digital Interface
- MIDI is a **scripting language** — it codes “events” that stand for the production of sounds. E.g., a MIDI event might include values for the pitch of a single note, its duration, and its volume.
- MIDI is a **standard** adopted by the electronic music industry for controlling devices, such as synthesisers and sound cards, that produce music.
- The MIDI standard is supported by most synthesisers, so sounds created on one synthesiser can be played and manipulated on another synthesiser and sound reasonably close.
- Computers must have a special MIDI interface, but this is incorporated into most sound cards. The sound card must also have both D/A and A/D converters


MIDI IN
MIDI OUT
SYNTHESIZER A
MIDI IN
MIDI OUT
MIDI IN
MIDI OUT
MIDI IN
MIDI OUT
SYNTHESIZER B
MIDI OUT
MIDI IN
MIDI IN
MIDI OUT
MASTER COMPUTER
OR SEQUENCER
MIDI IN
MIDI OUT
SYNTHESIZER C
MIDI IN
MIDI OUT
DRUM MACHINE

Brief history of MIDI

Before MIDI, synthesisers and electronic musical instruments from different manufacturers couldn't communicate with each other. Each brand had its own proprietary interface, limiting interoperability.

- **1981**: **The Concept of MIDI**

Dave Smith (Sequential Circuits) and Ikutaro Kakehashi (Roland) proposed a universal digital communication protocol for electronic instruments.

- **1983: Official Launch of MIDI 1.0**
- **1980s–1990s**: **MIDI Becomes Industry Standard**

Major companies (Yamaha, Korg, Roland) adopted MIDI. MIDI sequencers, DAWs, and samplers revolutionised music production.

- **2000s–2020s**: **MIDI 2.0 and Modern Advances**

MIDI 2.0 was announced in 2020, adding higher resolution, bidirectional communication, and more expressive controls.

MIDI is now used beyond music in lighting, robotics, gaming, and VR.

Role of MIDI in music production

- **Recording & Composition**
  - Notes, velocity, timing, and expression can be adjusted after recording.
  - Unlike audio recordings, MIDI sequences can control any virtual or hardware instrument.
- **Sound Design & Synthesis**
  - Enables automation of effects, such as modulation, pitch bends, and filter sweeps.
- **Editing & Arrangement**
  - Notes can be quantised, transposed, or adjusted in duration without affecting sound quality.

Role of MIDI in music production (cont.)

- **Live Performance & Control**
  - MIDI mapping allows musicians to control DAWs, lighting, and effects in live settings.
- **Synchronisation & Automation**
  - MIDI syncs multiple instruments and software using MIDI Clock or MIDI Timecode (MTC).
  - Automates tempo changes, effect parameters, and mixing settings in a DAW.
- **Music Notation & Scoring**
  - MIDI data can be converted into sheet music for composers and arrangers.

What is sound synthesis?

- Sound synthesis is the process of **generating sound electronically** using mathematical models, oscillators, or sampled audio.
- There are **various types of sound synthesis**: Subtractive Synthesis, Additive Synthesis, Frequency Modulation, Granular Synthesis, Wavetable Synthesis, Sampling-Based Synthesis, Physical Modeling Synthesis.
- **Applications of Sound Synthesis:**
  - **Music Production** – Synthesisers, DAWs, and VST (Virtual Studio Technology) plugins.
  - **Speech Synthesis** – Text-to-speech systems (Siri, Google Assistant).
  - **Game Audio** – Sound design in games.
  - **Film & Sound Design** – Creating sound effects for movies and animations.

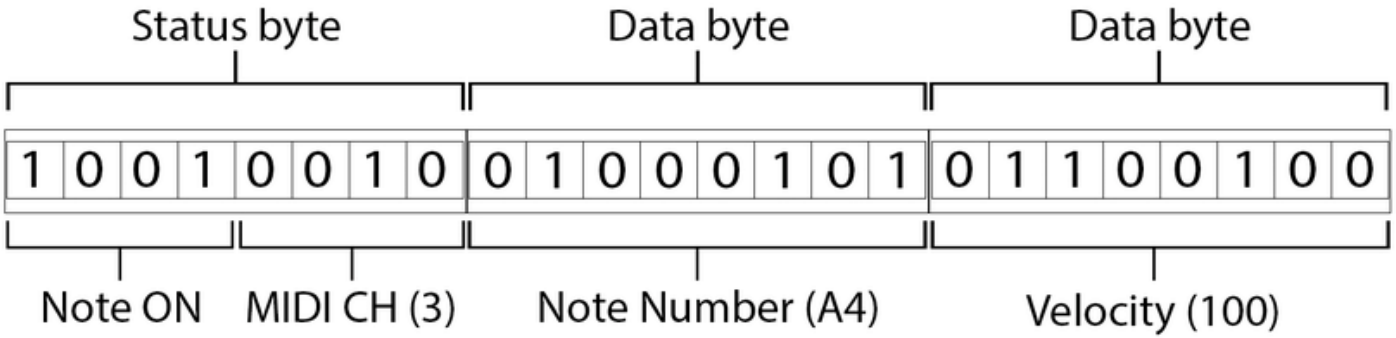
Agenda

- Overview of MIDI and Sound Synthesis
- Understanding MIDI Messages
- Hardware Aspects of MIDI
- Basics of Sound Synthesis
- Types of Synthesis

MIDI Messages

Each MIDI message consists of 1 status byte followed by 1 or 2 data bytes.

- The **status byte** defines the type of information being sent. The most significant half contains the actual **MIDI command**, and the second half contains the **MIDI channel** for which the command is for.
- **Data bytes** carry additional information (e.g., pitch, velocity, controller value).



MIDI Command

MIDI Channel

Example of MIDI Message

Binary: 1001 0000 | 0011 1100 | 0111 1111

Hex: 0x90 0x3C 0x7F

MIDI command = Note On (1001 or 0x9) & Channel 1 (0000) |

Note Number (0011 1100 or 0x3C = 60 for Middle C) |

Velocity (0111 1111 or 0x7F = 127 max velocity)

Hexadecimal (reminder)

Hexadecimal (or base 16) is a number system that uses 16 symbols:

- Digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Letters: A, B, C, D, E, F (which represent 10 to 15 in decimal)

- Each digit in a hexadecimal number represents a power of 16.
- Example: Let's take 2F3 (hex) and convert it to decimal:

2 times 16^2 + F times 16^1 + 3 times 16^0 = (2×256)+(F×16)+(3×1)

Since F = 15 in decimal, this becomes:

(2×256)+(15×16)+(3×1) =512+240+3= =755 (decimal)

Number Base Equivalences

dec	hex	bin
0	0	0
1	1	1
2	2	10
3	3	11
4	4	100
5	5	101
6	6	110
7	7	111

dec	hex	bin
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

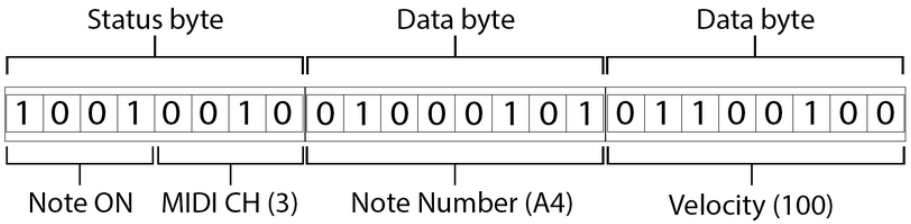
Why Use Hexadecimal?

1. Compact Representation: Hex is shorter than binary. For example:

   Binary: 1101 1110 1001

   Hex: DE9

2. Easy Conversion from Binary: **Every 4 binary digits = 1 hex digit**



3. Used for:

   Memory addresses (e.g., 0xFF3A)

   Colours in HTML/CSS (#FF6600 = orange)

   MIDI and digital audio data

Exercise

Exercise 1: Convert Hex to Decimal

A3

1F4

7D

B6

Exercise

Exercise 2: Convert Decimal to Hex

255

450

123

999

Exercise

Exercise 3: Convert Binary to Hex

0011 1100

1111 0001

1001 0110

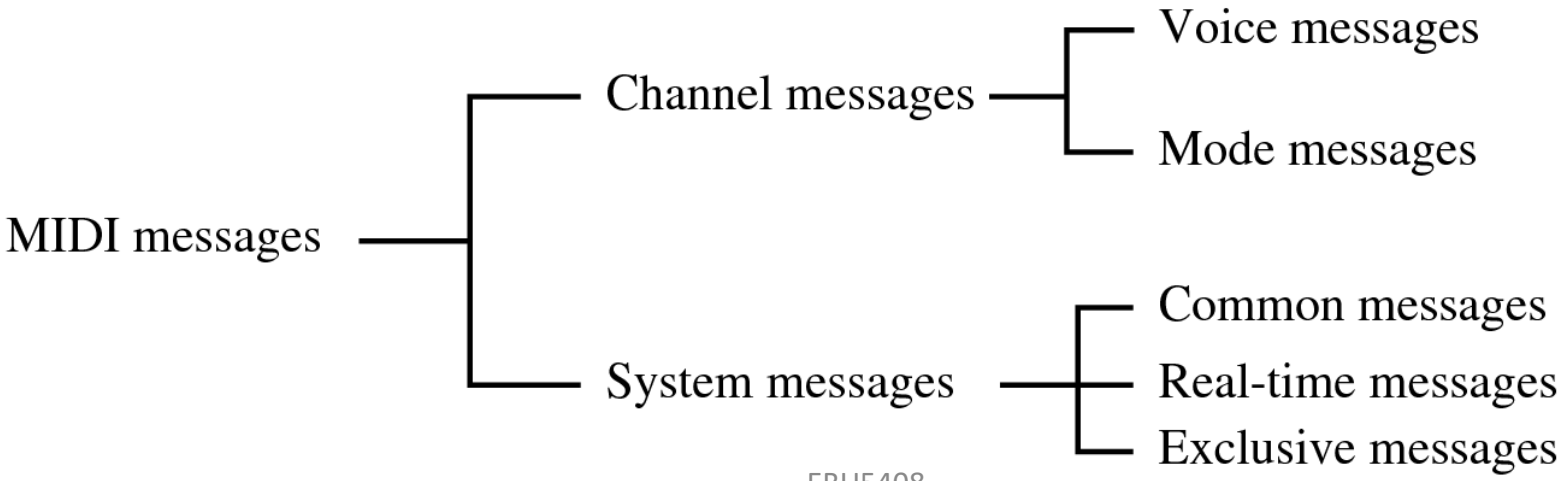
1101 1011

Types of MIDI messages

MIDI messages can be classified into two types: channel messages and system messages.

The messages from 0x80 to 0xEF are called **Channel Messages** because the second four bits of the command (or status byte) specify which channel the message affects.

The messages from 0xF0 to 0xFF are called **System Messages**; they do not affect any particular channel.



Channel Messages

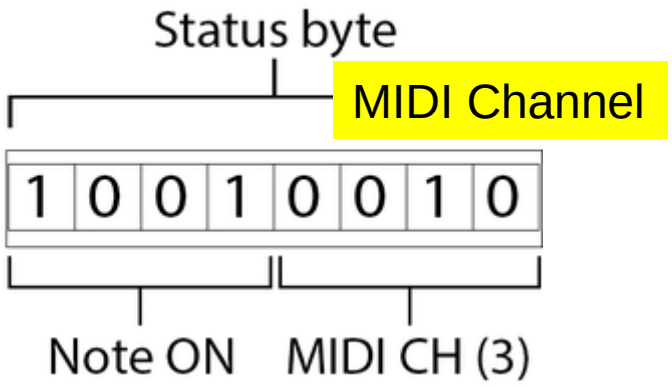
Channel Messages (sent to a specific MIDI channel)

- Note On (**0x9**0 - 0x9F) → Starts a note (includes pitch & velocity).
- Note Off (**0x8**0 - 0x8F) → Stops a note.
- Control Change (**0xB**0 - 0xBF) → Adjusts parameters (e.g., volume, modulation).
- Program Change (**0xC**0 - 0xCF) → Switches instrument patches.
- Pitch Bend (**0xE**0 - 0xEF) → Bends the pitch of a note.

MIDI Channels

- There are **16 channels** numbered from 1 to 16. The channel forms the last 4 bits (the least significant bits) of the Status byte. (Channel 1 is coded 0000 or 0x0, and Channel 16 is coded 1111 or 0xF.)

- Usually, a channel is associated with a particular instrument: e.g., channel 1 is for piano, channel 10 is for drums, etc. But one can switch instruments midstream, if desired, and associate another instrument with any channel.



Channel Voice Messages

Voice messages

- This type of channel message controls a voice, i.e., sends information specifying which note to play or to turn off, and encodes key pressure.
- Voice messages are also used to specify controller effects such as sustain, vibrato, tremolo, and the pitch wheel.

Channel Voice Messages (cont.)

Voice Message	Status Byte	Data Byte1	Data Byte2
Note Off	0x8n	Key number	Note Off velocity
Note On	0x9n	Key number	Note On velocity
Poly. Key Pressure	0xAn	Key number	Amount
Control Change	0xBn	Controller num.	Controller value
Program Change	0xCn	Program number	None
Channel Pressure	0xDn	Pressure value	None
Pitch Bend	0xEn	MSB	LSB

0x indicates hexadecimal, and ‘n’ in the status byte hex value stands for a channel number. All values are in 0..127 except Controller number, which is in 0..120

Common Channel Voice Messages

- 0x90 - 0x9F: Note On. Initiates the playing of a note on a specific MIDI channel.
- Followed by a note number (0-127) and a velocity value (0-127).

- (0x80 - 0x8F): Note Off. Stops the playing of a note on a specific MIDI channel.
- Followed by a note number and a velocity value.

- (0xC0 - 0xCF): Program Change. Selects a specific instrument or sound (program) on a specific MIDI channel.
- Followed by a program number (0-127).

- (0xE0 - 0xEF): Pitch Bend. Controls the pitch bend of a specific MIDI channel.
- Followed by two 7-bit values representing the pitch bend amount.

Exercise 1

You receive the following MIDI message in hexadecimal format:

0x90 0x3C 0x64

Questions:

1. What type of MIDI message is this (MIDI command)?
2. What is the note number (in decimal)?
3. What is the velocity of the note (in decimal)?
4. What MIDI channel is this message on?

Exercise 2

Write the MIDI message (in hexadecimal) to turn off a D4 (MIDI note 62) on Channel 5 with a velocity of 50.

Exercise 3

Consider the following MIDI messages:

0x91 0x40 0x7F

0x92 0x45 0x64

0x93 0x48 0x50

Questions:

1. What is the difference between these messages?
2. Which notes are being played, and on which MIDI channels?
3. Which note has the highest velocity?

Exercise 4

Write a sequence of MIDI messages to:

- Start playing G3 (MIDI note 55) on Channel 7 with a velocity of 90.
- Hold the note for a duration.
- Stop playing the note with a velocity of 0.

Exercise 5

You extract the following MIDI event sequence from a file:

0x90 0x3C 0x50

0x80 0x3C 0x00

0x90 0x40 0x64

0x90 0x43 0x64

Questions:

1. What happens in this MIDI sequence?
2. Identify the notes and describe their interaction.

Channel Mode Messages

- A Channel Mode message has its first data byte in 121 through 127 (0x79–7F).
- Channel mode messages determine how an instrument processes MIDI voice messages: respond to all messages, respond just to the correct channel, don’t respond at all, or go over to local control of the instrument.

Channel Mode Messages (cont.)

1st Data Byte	Description	Meaning of 2nd Data Byte
0x79	Reset all controllers	None; set to 0
0x7A	Local control	0 = off; 127 = on
0x7B	All notes off	None; set to 0
0x7C	Omni mode off	None; set to 0
0x7D	Omni mode on	None; set to 0
0x7E	Mono mode on (Poly mode off)	Controller number
0x7F	Poly mode on (Mono mode off)	None; set to 0

System Messages

- System messages have no channel number — commands that are not channel specific, such as timing signals for synchronisation, positioning information in pre-recorded MIDI sequences, and detailed setup information for the destination device.
- Opcodes for all system messages start with **0xF**.
- System messages are divided into three classifications, according to their use: **common, real-time, and exclusive system messages**.

System Common Messages

These messages provide essential information for all devices in the system, such as song positioning and timing.

Status Byte (Hex)	Message Name	Description
0xF0	System Exclusive (SysEx) Start	Begins a SysEx message for manufacturer-specific data.
0xF1	MIDI Time Code Quarter Frame	Synchronizes time code across MIDI devices.
0xF2	Song Position Pointer	Indicates a position in a MIDI sequence in 16th note increments.
0xF3	Song Select	Selects a specific song in a sequencer.
0xF6	Tune Request	Requests tuning of analog synthesizers.
0xF7	End of SysEx (EOX)	Marks the end of a SysEx message.

System Real-time Messages

These messages are used for synchronisation and timing, ensuring devices play together in real-time.

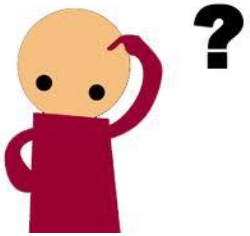
Status Byte (Hex)	Message Name	Description
0xF8	Timing Clock	Sent 24 times per quarter note for tempo synchronization.
0xFA	Start	Starts playback of a MIDI sequence.
0xFB	Continue	Resumes playback from the last stopped position.
0xFC	Stop	Stops playback of a sequence.
0xFE	Active Sensing	Detects if a connection is still active.
0xFF	System Reset	Resets all MIDI devices.

System Exclusive Messages

These messages are included so that the MIDI standard can be extended by manufacturers.

- After the initial code, a stream of any specific messages can be inserted that apply to their own product.
- A System Exclusive message is supposed to be terminated by a terminator byte 0xF7.
- The terminator is optional, and the data stream may simply be ended by sending the status byte of the next message.

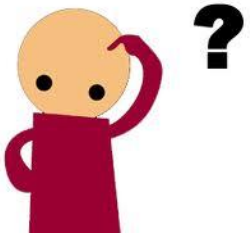
Question 1



Below are several hexadecimal MIDI messages. Identify whether each one is a System Common Message or System Real-Time Message and explain its function.

0xF8
0xF2
0xFA
0xFE

Question 2

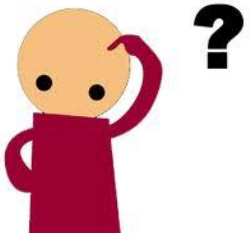


A MIDI device receives the following sequence of messages:

0xFA → 0xF8 → 0xF8 → 0xF8 → 0xFC

What is happening in this sequence?

Question 3



A MIDI keyboard sends 0xFE every 300ms.

1. What does this message do?
2. What happens if the receiving device stops receiving this message?

Responding to MIDI channel messages

- The way a synthetic musical instrument responds to a MIDI message is usually by simply ignoring any play sound message that is not for its channel.
- If several messages are for its channel, then the instrument responds, provided it is **multi-voice**, i.e., can play more than a single note at once.

Voice and Timbre

- The term **timbre** is MIDI terminology for what instrument it is trying to emulate, e.g. a piano as opposed to a violin: it is the quality of the sound.
  - An instrument (or sound card) that is **multi-timbral** is one that is capable of playing many different sounds at the same time, e.g., piano, brass, drums, etc.
- The term **voice** is used in MIDI to mean every different timbre and pitch that the tone module can produce at the same time.
- Different timbres are produced digitally by using a **patch** — the set of control settings that define a particular timbre. Patches are often organized into databases, called **banks**.

General MIDI

General MIDI is a scheme for standardising the assignment of instruments to patch numbers.

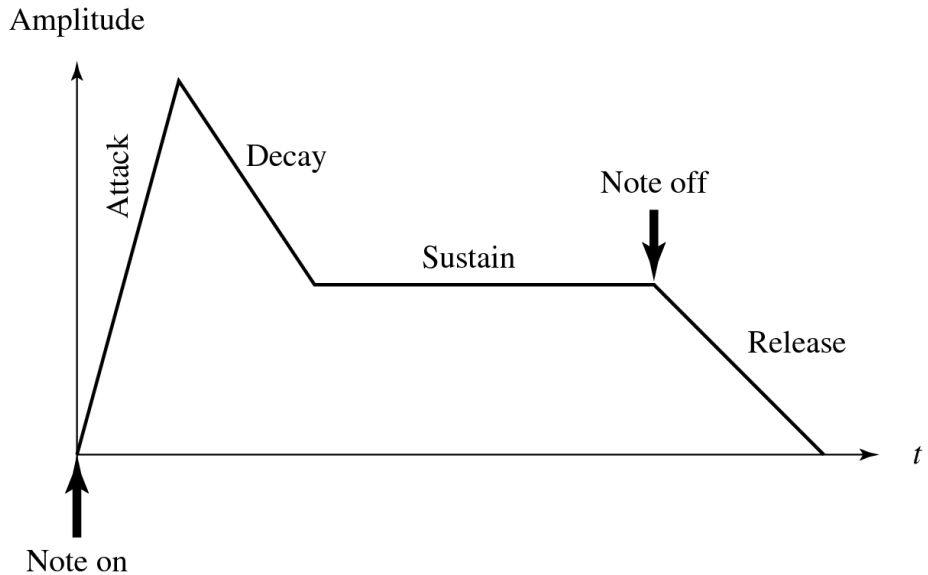
- Channel 10 is reserved for percussion instruments, and there are 128 patches associated with standard instruments.
- For most instruments, a typical message might be a Note On message (meaning, e.g., a keypress and release), consisting of what channel, what pitch, and what “velocity” (i.e., volume).
- For percussion instruments, however, the pitch data means which kind of drum.
- A standard percussion map specifies 47 percussion sounds.
- Where a “note” appears on a musical score determines what percussion instrument is being struck: a bongo drum, a cymbal.

General MIDI (cont.)

- A Note On message consists of “status” byte — which channel, what pitch — followed by two data bytes. It is followed by a Note Off message, which also has a pitch (which note to turn off) and a velocity (often set to zero).
- Other requirements for General MIDI compatibility:
  - MIDI device must support all 16 channels;
  - a device must be multitimbral (i.e., each channel can play a different instrument/program);
  - a device must be polyphonic (i.e., each channel is able to play many voices);
  - and there must be a minimum of 24 dynamically allocated voices.

General MIDI (cont.)

- A MIDI device often is capable of **programmability**, and also can change the **envelope** describing how the amplitude of a sound changes over time.
- The figure shows a model of the response of a digital instrument to a Note On message:



Stages of amplitude versus time for a music note

Agenda

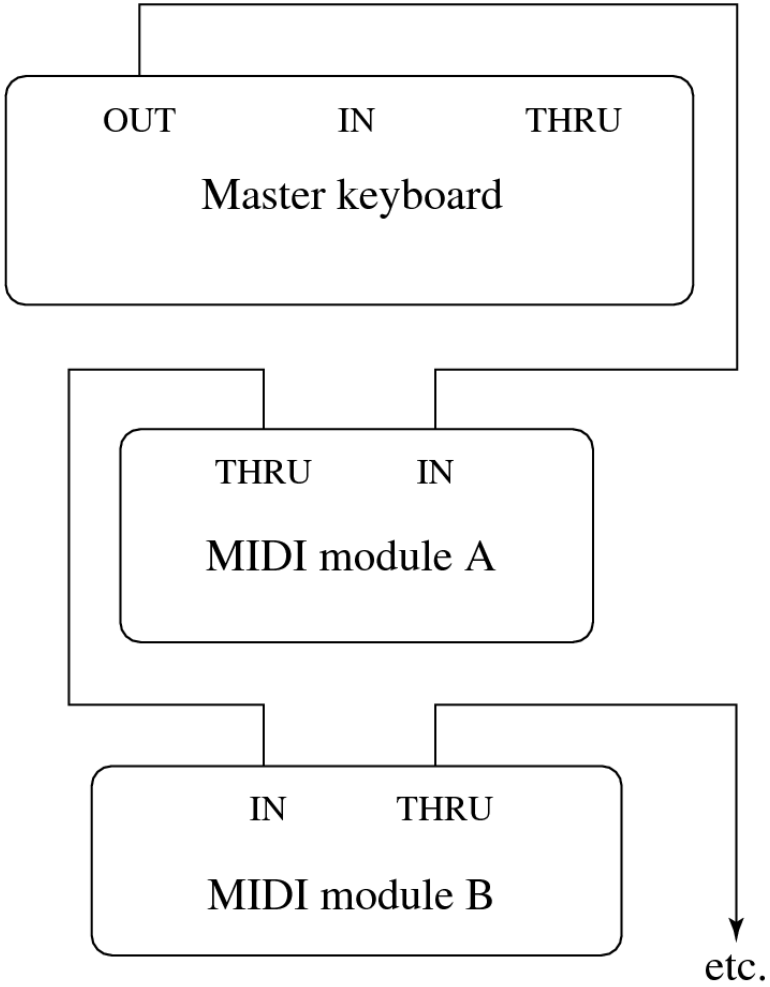
- Overview of MIDI and Sound Synthesis
- Understanding MIDI Messages
- Hardware Aspects of MIDI
- Basics of Sound Synthesis
- Types of Synthesis

MIDI ports

The physical MIDI ports consist of 5-pin connectors for IN and OUT, as well as a third connector called THRU.

- MIDI communication is half-duplex, i.e., data can flow in both directions (from a MIDI controller to a synthesiser, for example), but not simultaneously. At any given time, a device can either transmit or receive MIDI messages, but not at the same moment.
- MIDI IN is the connector via which the device receives all MIDI data.
- MIDI OUT is the connector through which the device transmits all the MIDI data it generates itself.
- MIDI THRU is the connector by which the device echoes (forwards) the data it receives from MIDI IN. Note that it is only the MIDI IN data that is echoed by MIDI THRU — all the data generated by the device itself is sent via MIDI OUT.

A Typical MIDI Setup



MIDI ports: Practical Example

Imagine you have the following setup:

1. A MIDI keyboard (controller)

2. A synthesiser

3. A drum machine

<u>Connections</u>:

- Connect the MIDI OUT of the keyboard to the MIDI IN of the synthesiser.

- Connect the MIDI THRU of the synthesiser to the MIDI IN of the drum machine.

<u>Result</u>: When you play the keyboard, it sends MIDI data to the synthesiser, which plays the notes. The same MIDI data is relayed through the synthesiser's MIDI THRU port to the drum machine, which also responds to the notes.

MIDI sequencer

A MIDI sequencer is a device or software application that records, edits, and plays back MIDI data.

MIDI data allows musicians to arrange and modify their compositions without affecting the quality of the sounds being generated.

Examples of MIDI Sequencers:

- Software-based: Ableton Live, FL Studio, Logic Pro, Cubase
- Hardware-based: Akai MPC series, Roland MC-707

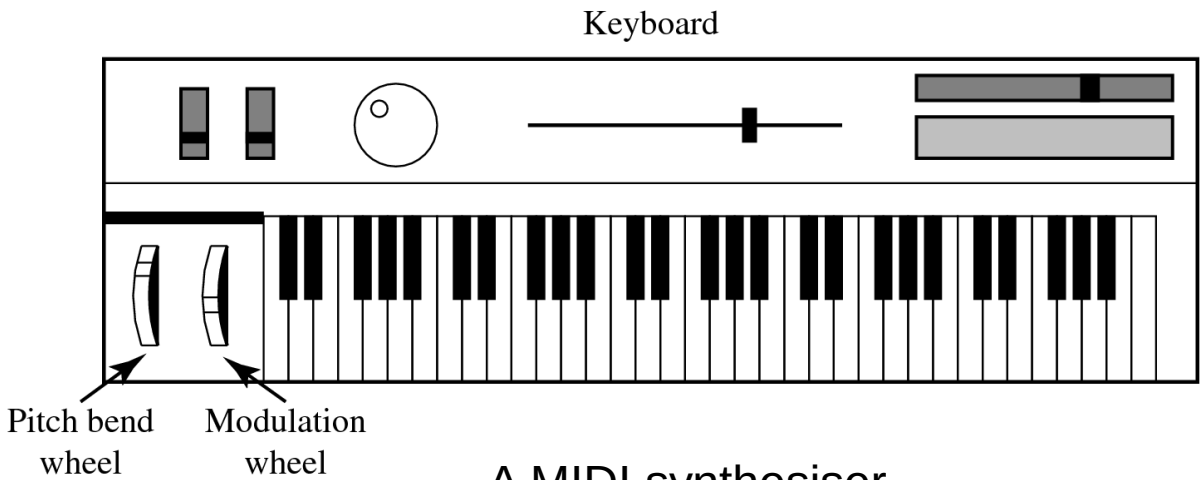
Functions of a MIDI sequencer

- Recording MIDI Data – Captures MIDI messages from a keyboard, drum pad, or other MIDI controller.
- Editing MIDI Data – Allows modification of note timing, pitch, duration, and velocity using a piano roll or event list.
- Looping & Layering – Repeats MIDI patterns and layers multiple tracks for complex arrangements.
- Playback & Integration – Sends MIDI data to software synthesisers, hardware sound modules, or virtual instruments.
- Automation – Controls changes in volume, modulation, pitch bend, and other parameters over time.

MIDI Synthesiser

A MIDI synthesiser is a musical instrument that uses the MIDI protocol to **generate sounds**.

A MIDI synthesiser converts MIDI data (which contains instructions about musical notes, pitch, and timing) into audio signals.



A MIDI synthesiser

How a MIDI Synthesiser Works

- MIDI Data Input: A MIDI synthesiser receives MIDI signals from a MIDI controller (such as a keyboard, drum pad, or other MIDI-enabled devices).
- Sound Generation: The synthesiser uses this MIDI data to control sound-generating hardware or software. It can produce a wide variety of sounds, including piano, strings, drums, and electronic effects. The sound is created by a sound engine (analog or digital) inside the synthesiser.
- Audio Output: Once the MIDI data is processed, the synthesiser outputs the generated sound through speakers or audio interfaces.

MIDI synthesisers are widely used in studios, electronic music production, and live performances.

Agenda

- Overview of MIDI and Sound Synthesis
- Understanding MIDI Messages
- Hardware Aspects of MIDI
- Basics of Sound Synthesis
- Types of Synthesis

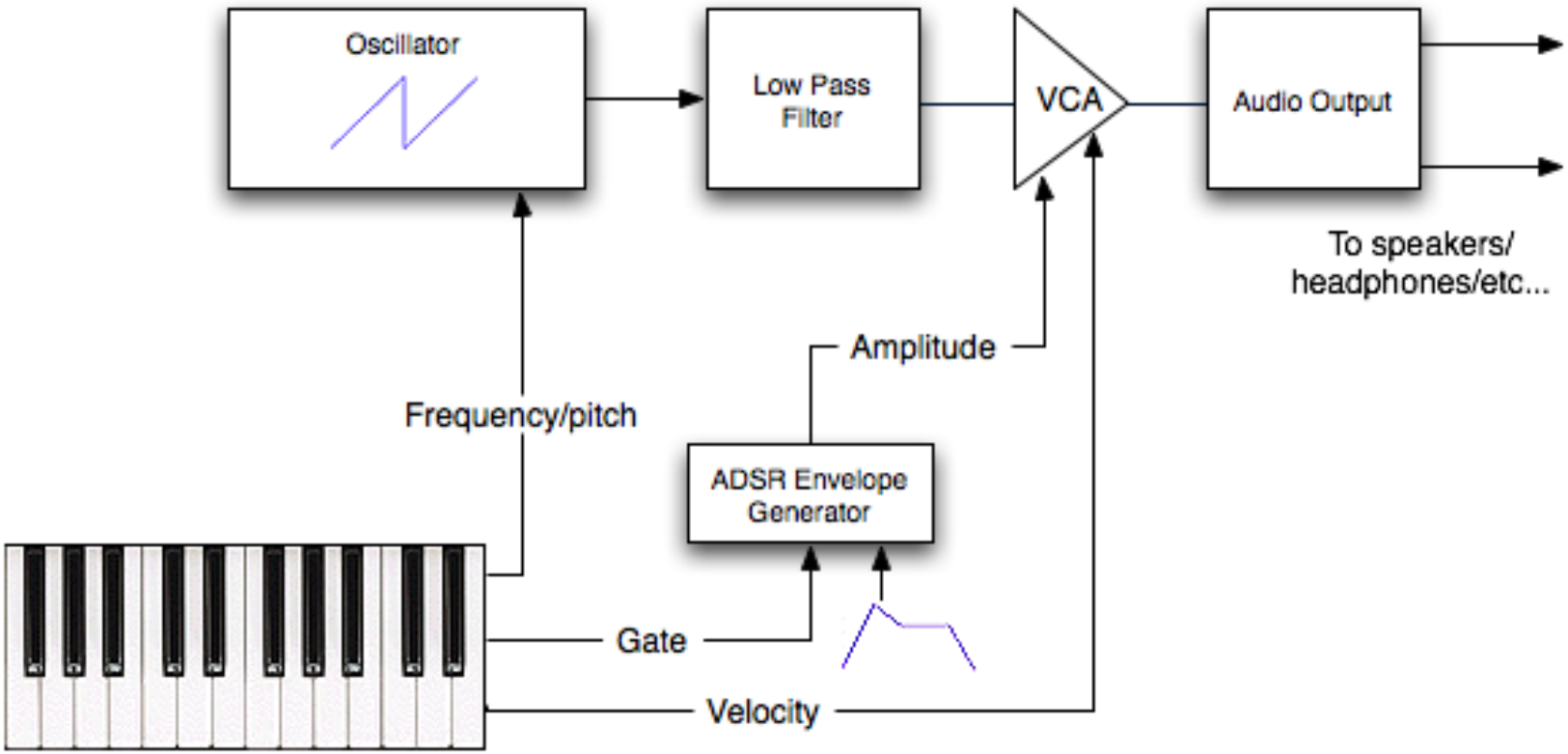
Key Elements of Synthesisers

A sound synthesiser system generates and manipulates sound electronically. It consists of several key elements that work together to shape and control the sound.

Below are the basic elements of a synthesiser:

- **Oscillators**: generate a consistent, repeating signal (Sine, Square, Sawtooth, Triangle, etc.)
- **Filters**: Low-pass, High-pass, Band-pass
- **Amplifier**
- **Envelope Generator** (ADSR): Attack, Decay, Sustain, Release
- **Modulation effects** (vibrato, tremolo, wobble)

Key Elements of Synthesisers

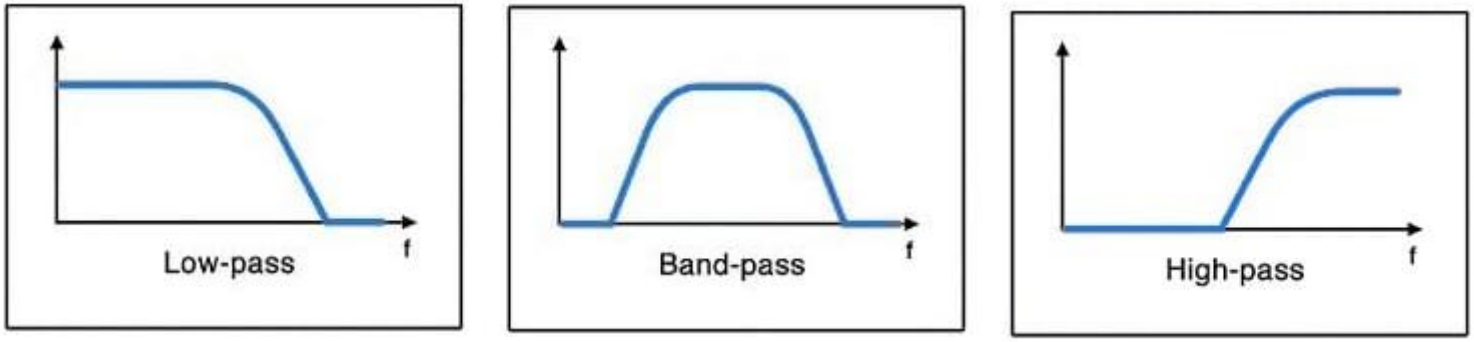


Oscillators

- An oscillator generates a consistent, repeating signal, i.e. the raw sound waveforms (sine, square, sawtooth, triangle, noise).
- It determines the pitch and timbre of the sound.
- Multiple oscillators can be used to create richer sounds.

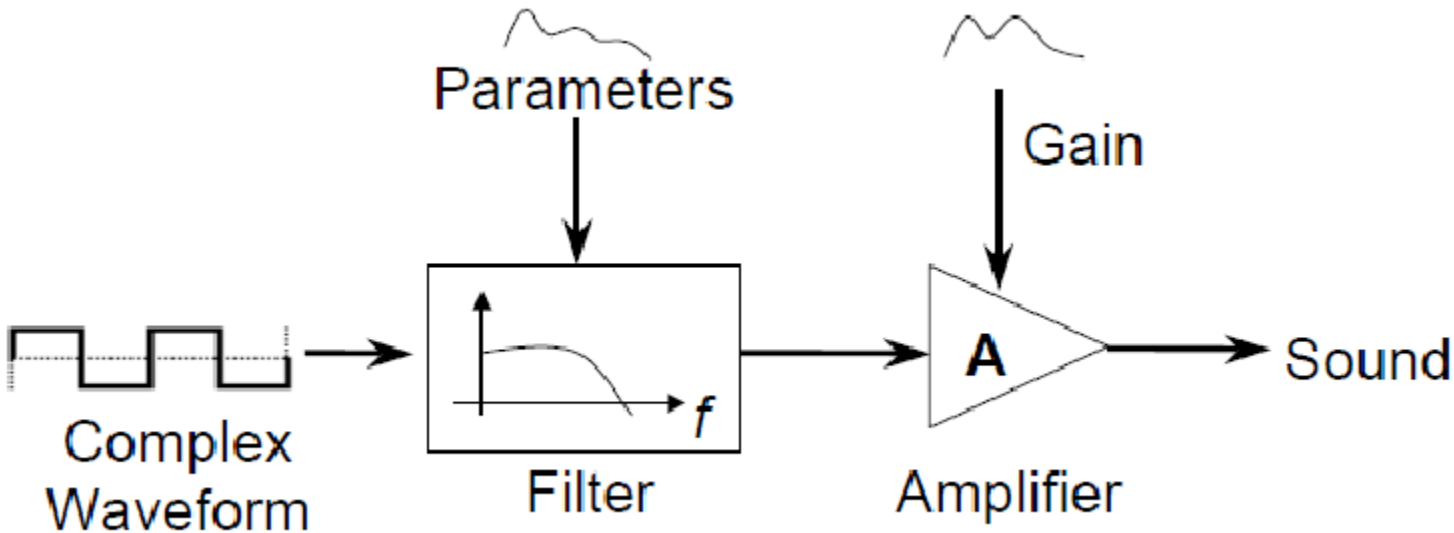
Filters

- Shape the frequency content of the sound.
- Remove or emphasise specific frequencies (e.g., low-pass, high-pass, band-pass filters).
- Essential for sound sculpting (e.g., subtractive synthesis).



Amplifier

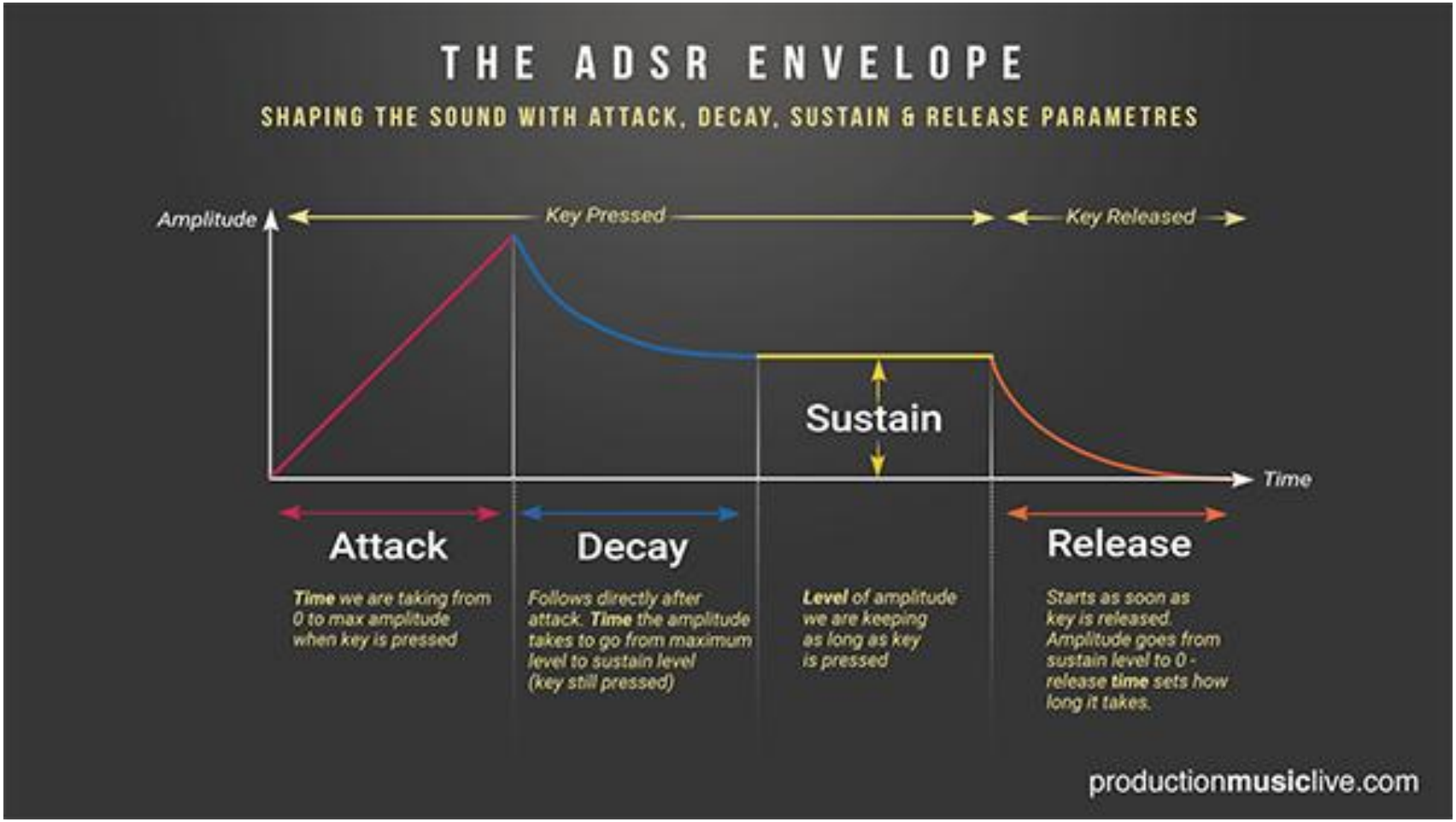
- Controls the loudness (amplitude) of the sound over time.
- Works with an envelope generator to create dynamics.



Envelope Generator

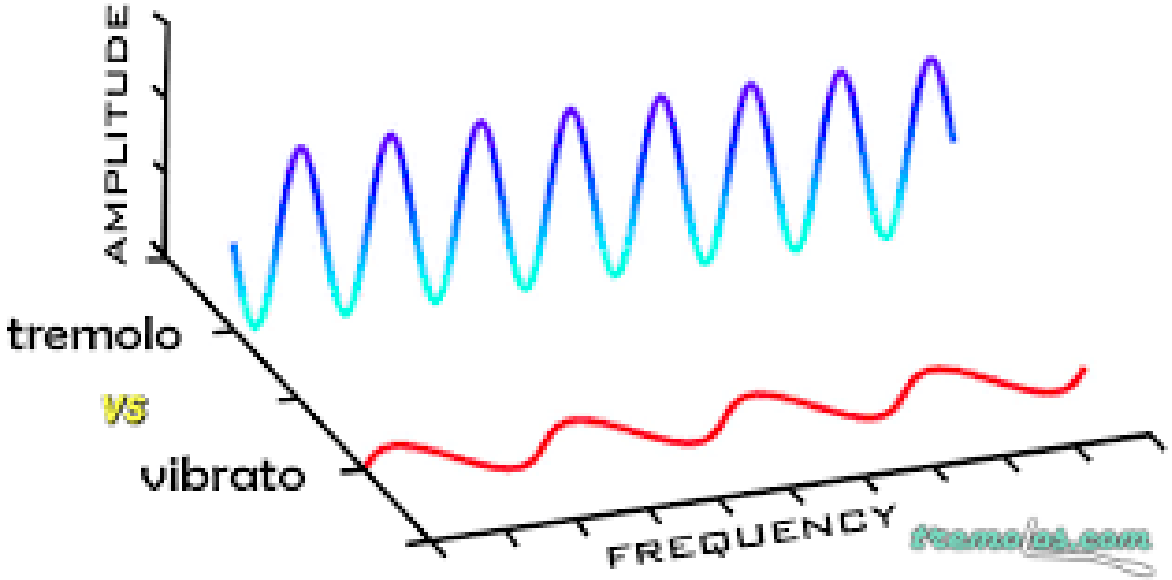
- Modulates how a sound evolves over time.
- Controls amplitude, filter, or pitch using four stages:
  - Attack (time to reach peak)
  - Decay (time to settle after peak)
  - Sustain (level held while key is pressed)
  - Release (fade out after key is released)

Envelope Generator (cont.)

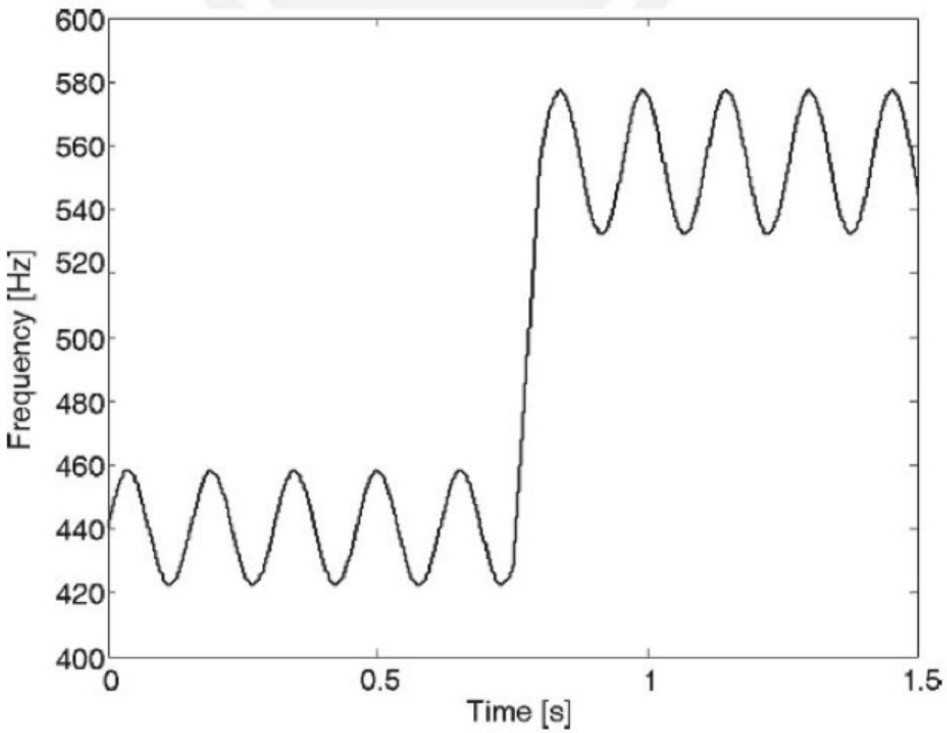


Modulation effects

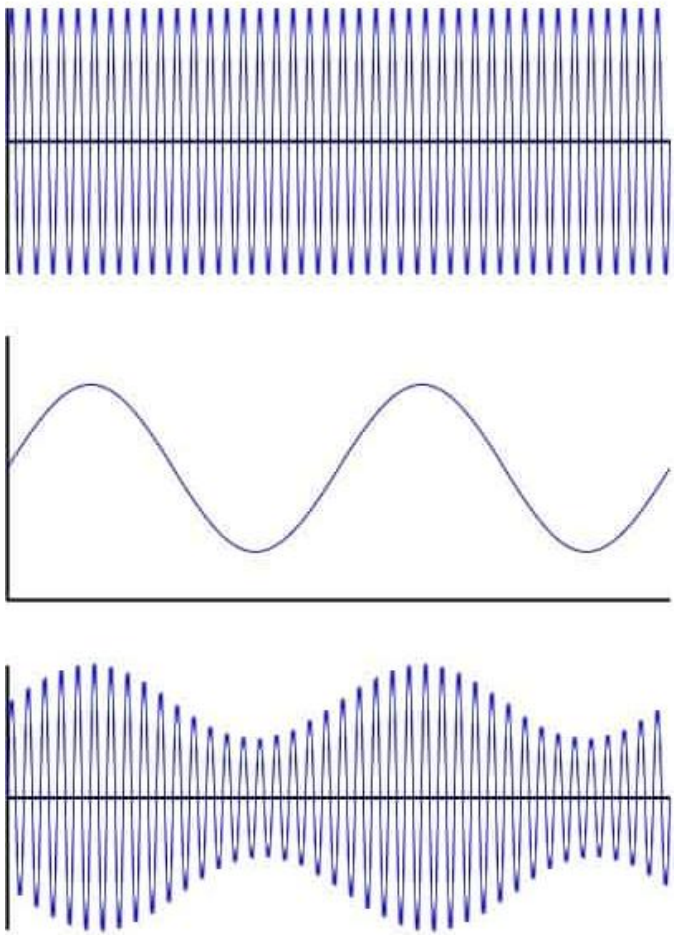
- Modulates parameters like pitch, filter cutoff, and volume.
- Creates vibrato (pitch modulation) or tremolo (amplitude modulation).



Vibrato vs Tremollo



Vibrato



Tremollo

Basics of Sound Synthesis

- Oscillators generate the raw sound.
- Filters and Amplifiers sculpt the tone and dynamics.
- Envelope generators and Modulators add movement and expression.
- The MIDI interface connects external controllers.
- Finally, the output stage ensures the sound is delivered cleanly and at proper levels.

Sound Synthesis: Practical Example

Example: **Creating a Bass Sound**

- Oscillator: Start with a sawtooth wave for a bright, rich tone.
- Filter: Apply a low-pass filter to remove high frequencies and create warmth.
- Envelope: Set a short attack, medium decay, low sustain, and short release for a punchy sound.
- Modulation: Add a slight vibrato for expressiveness.

Example: Synthesising a Simple Sound in MATLAB

Task 1: Generate a Basic Waveform

1. Create a sine wave of 440 Hz (A4 note).
2. Sample rate: 44100 Hz, duration: 2 seconds.

Task 2: Create an ADSR Envelope

Define an Attack, Decay, Sustain, and Release (ADSR) envelope.

💡 Hint: Use linspace() and ones() to shape different envelope phases.

```
fs = 44100; % Sampling rate
t = 0:1/fs:2; % Time vector (2 seconds)
f = 440; % Frequency of A4

% Task 1: Generate a Sine Wave
wave = sin(2 * pi * f * t);

% Task 2: Create an ADSR Envelope
A = linspace(0, 1, fs * 0.15); % Attack (150ms)
D = linspace(1, 0.7, fs * 0.25); % Decay (250ms)
S = 0.7 * ones(1, fs * 1); % Sustain (1s)
R = linspace(0.7, 0, fs * 0.4); % Release (400ms)
envelope = [A, D, S, R];
```

Example: Synthesising a Simple Sound in MATLAB (cont.)

Task 3: Apply Frequency Modulation and the envelop

1. Create a modulator wave (e.g., sine wave at 5 Hz).
2. Use the modulator to vary the frequency of the main waveform.

💡 Hint: Use sin(2 * pi * f * t + mod_index * sin(2 * pi * mod_freq * t)).

3. Multiply the waveform by the envelope.

Task 4: Play and Save the Sound

1. Use soundsc() to play the sound.
2. Use audiowrite() to save it as a .wav file.

```
% Task 3: Apply Frequency Modulation
mod_freq = 5; % 5 Hz modulation
mod_index = 50; % Modulation depth
wave = sin(2 * pi * f * t + mod_index * sin(2 * pi * mod_freq * t));

% Apply Envelope
wave = wave(1:length(envelope)) .* envelope;

% Task 4: Play and Save
soundsc(wave, fs);
audiowrite('synthesized_sound.wav', wave, fs);
```

Agenda

- Overview of MIDI and Sound Synthesis
- Understanding MIDI Messages
- Hardware Aspects of MIDI
- Basics of Sound Synthesis
- Types of Synthesis

Types of Synthesis

- **Subtractive Synthesis**: starts with a rich waveform (saw, square, noise) and removes frequencies using filters.
- **Additive Synthesis**: builds complex sounds by adding sine waves at different frequencies (harmonics).
- **Frequency Modulation (FM) Synthesis**: uses one waveform (carrier) modulated by another (modulator) to create complex timbres
- **Wavetable Synthesis**: uses pre-recorded waveforms stored in a table and morphs between them.
- **Granular Synthesis**: breaks sound into small grains and reassembles them in creative ways.

Subtractive Synthesis

Concept:

Subtractive synthesis is a form of music synthesis in which rich waveforms are generated by oscillators and then shaped using components such as filters and envelopes.

Key Elements:

- Oscillator (VCO): Generates raw waveforms.
- Filter (VCF): Shapes the harmonic content.
- Amplifier (VCA): Controls loudness over time.
- Envelope Generator (ADSR): Controls volume, filter cutoff, etc.

Example:

- Start with a sawtooth wave (rich in harmonics).
- Apply a low-pass filter to remove high frequencies, creating a warm, mellow sound.

Additive Synthesis

Concept:

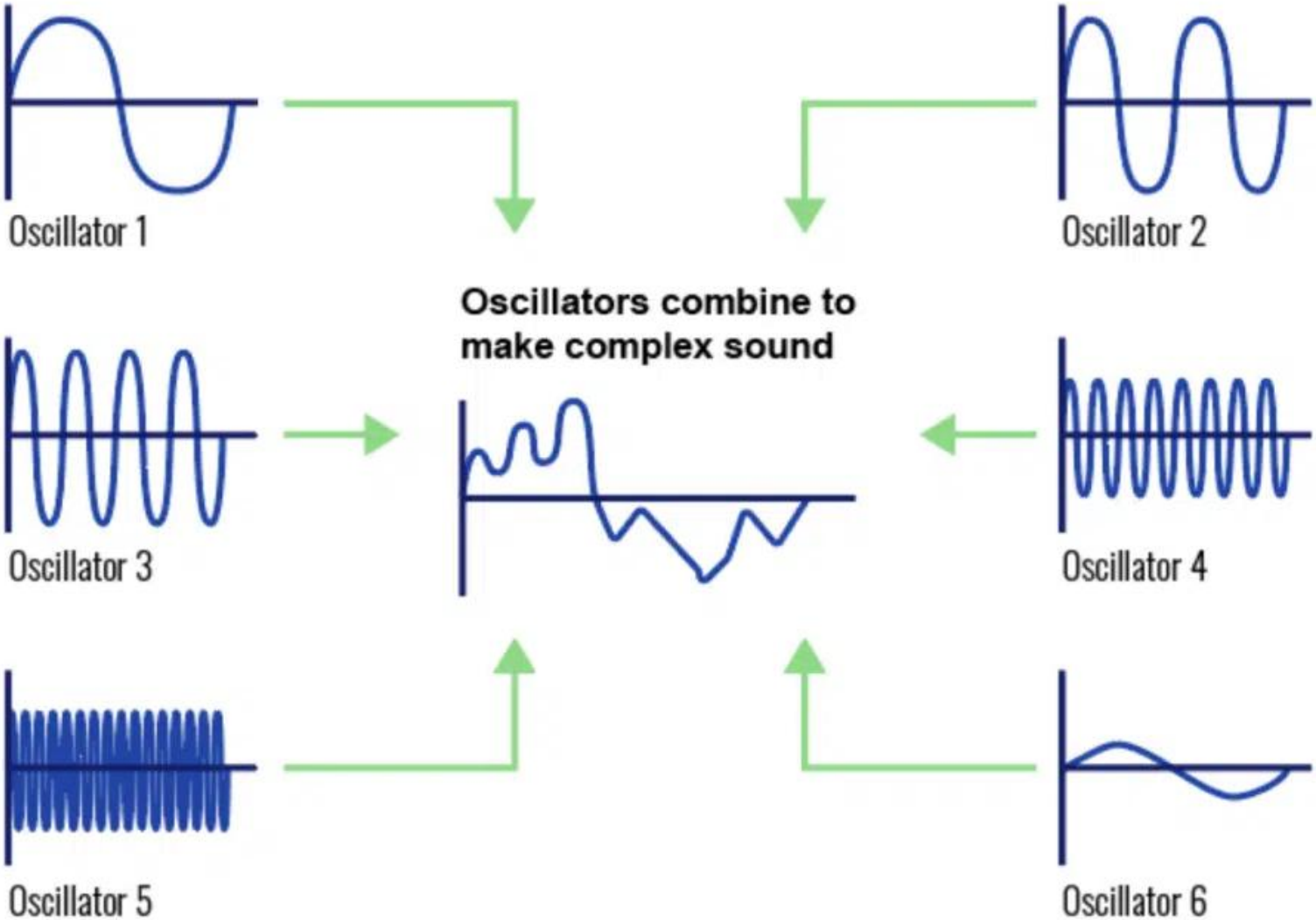
- Builds complex sounds by adding sine waves of different frequencies, amplitudes, and phases.
- Based on Fourier synthesis (any sound can be represented as the sum of sine waves).

Key Elements:

- Oscillators: Multiple sine waves combined to form complex tones.
- Amplitude Modulation: Adjusts each sine wave's loudness.

Example:

Combine sine waves at 440 Hz (fundamental), 880 Hz (2nd harmonic), 1320 Hz (3rd harmonic), etc., with varying amplitudes to create a bell-like sound.


Oscillator 1
Oscillator 2
Oscillators combine to make complex sound
Oscillator 3
Oscillator 4
Oscillator 5
Oscillator 6

Frequency Modulation

Concept:

- Uses one oscillator (carrier) whose frequency is modulated by another oscillator (modulator).
- Creates complex harmonics by changing the carrier’s pitch dynamically.

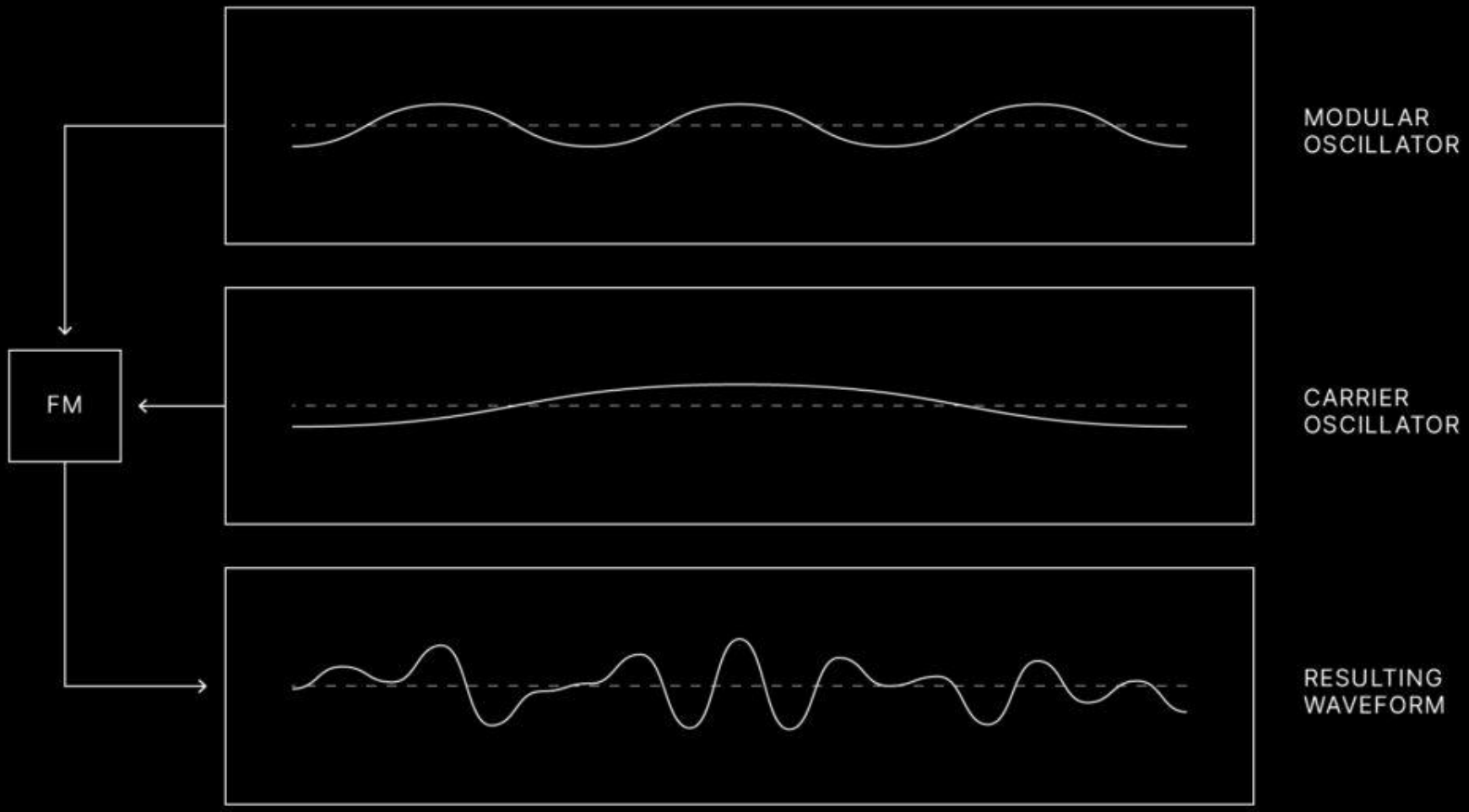
Key Elements:

- Carrier Oscillator: Defines the base frequency.
- Modulator Oscillator: Alters the frequency of the carrier.
- Modulation Index: Controls the depth of modulation.

Example:

Use a sine wave as the carrier (e.g., 440 Hz) and another sine wave as the modulator (e.g., 5 Hz). The modulator creates variations in the carrier's frequency, producing a rich, evolving timbre.

FM synthesis



Wavetable synthesis

Concept:

- Uses digital tables (arrays of waveforms) instead of simple oscillators.
- Morphs between different waveforms over time to create evolving sounds.

Key Elements:

- Wavetable Oscillator: Selects waveforms from a table.
- Interpolation: Blends between waves for smooth transitions.
- Modulation: LFOs, envelopes, and MIDI control movement between waves.

Example:

A wavetable contains waveforms ranging from a sine wave to a sawtooth wave. As the wavetable is scanned, the sound morphs smoothly from a sine wave to a sawtooth wave.

Granular synthesis

Concept:

- Splits a sound sample into tiny "grains" and rearranges them.
- Can time-stretch, pitch-shift, or create completely new textures.

Key Elements:

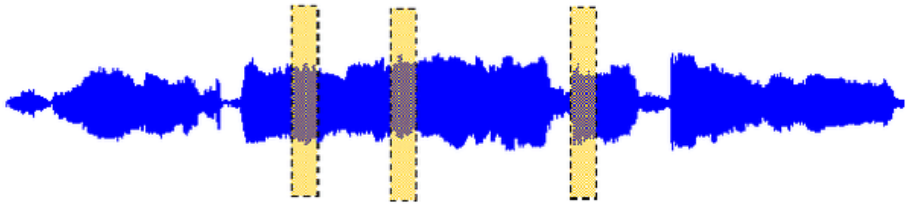
- Grain Size: Defines the length of individual grains.
- Playback Speed: Controls time-stretching and pitch.
- Randomisation: Changes position and envelope of grains.

Example:

Take a recording of a piano note. Split it into grains and rearrange them to create a shimmering, textured sound.

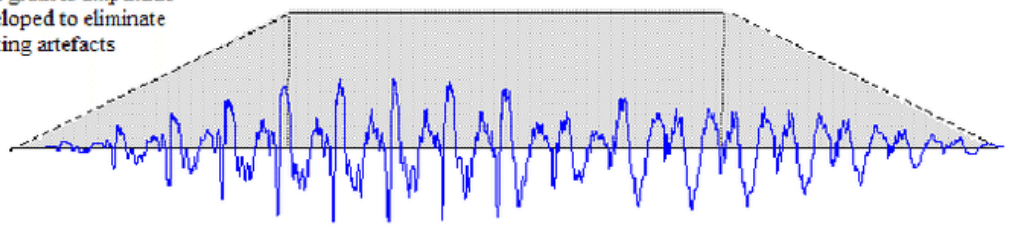
SELECTION

Grains are selected from source waveforms



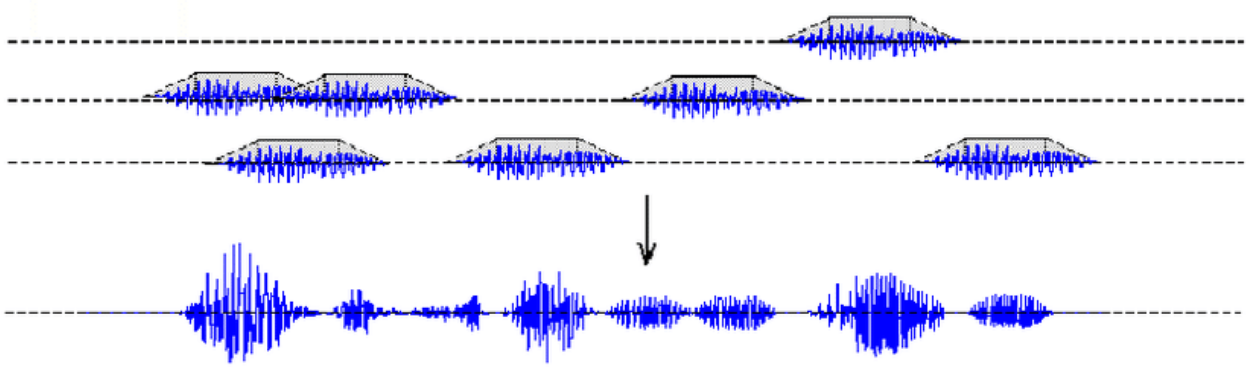
ENVELOPE

Each grain is amplitude enveloped to eliminate clicking artefacts



SUMMATION

A large number of grains, drawn from a number of sources are then summed to produce the output



Types of Synthesis: Comparison

Type	Approach	Strengths	Weaknesses
Subtractive	Start with rich waveforms, filter out frequencies.	Simple, intuitive, great for classic analog sounds.	Limited complexity in timbre.
Additive	Build sound by adding sine waves (harmonics).	Can create very complex and realistic sounds.	Computationally intensive, difficult to program.
FM	Modulate the frequency of one waveform with another.	Capable of creating bright, metallic, and complex sounds.	Can be difficult to control and predict.
Wavetable	Use and morph pre-recorded waveforms.	Dynamic and evolving sounds, great for modern music.	Limited by the quality of the wavetable.
Granular	Break sound into grains and reassemble.	Extremely flexible, great for experimental sound design.	Can sound unnatural if not used carefully.

Summary

- Overview of MIDI and Sound Synthesis
- Understanding MIDI Messages
- Hardware Aspects of MIDI
- Basics of Sound Synthesis
- Types of Synthesis